

Praktikum Pemrograman Berorientasi Objek

Pertemuan 4

Materi Pertemuan 4

Gambaran Materi Pembelajaran Hari Ini

Materi Pertemuan 5

01

Abstract Class

Mengenal class yang berbentuk abstrak

03

Polimorfisme

Mengenal sifat polimorfisme objek dari PBO

05

Casting

Mengenal istilah lainnya yang terdapat dalam polimorfisme

02

Interface

Mengenal interface sebagai bentuk lain dari abstract class

04

Operator Instanceof

Mengenal penggunaan operator instanceof



Abstract Class

Abstract Class

Keyword abstract dapat diterapkan pada class maupun method untuk menambahkan security pada program.

Abstract Class tidak dapat diinstansiasi (tidak bisa dibuat objeknya secara langsung menggunakan new), tetapi dapat diturunkan ke sebuah subclass.

Abstract Method dideklarasikan tanpa memiliki body / implementasi (tanpa tanda kurung kurawal {}).

Subclass yang mewarisi abstract class wajib mengimplementasikan (override) semua abstract method yang ada di dalamnya.

```
public abstract class Kendaraan {  
    String merek;  
  
    public abstract void maju();  
  
    public void berhenti() {  
        System.out.println("Kendaraan berhenti.");  
    }  
}
```

Abstract Class (lanjutan)

Contoh penggunaan abstract class

```
public abstract class Kendaraan {  
    String merek;  
  
    public abstract void maju();  
  
    public void berhenti() {  
        System.out.println("Kendaraan berhenti.");  
    }  
}
```

```
// Mobil wajib mengimplementasikan  
// method 'maju()' dari Kendaraan  
public class Mobil extends Kendaraan {  
  
    @Override  
    public void maju() {  
        System.out.println("Mobil maju.");  
    }  
}
```

```
public class Main {  
    public static void main(String[] args) {  
        Kendaraan myKendaraan = new Kendaraan();  
        // Error, tidak bisa meng-instansiasi kelas abstrak secara langsung:  
  
        Mobil myMobil = new Mobil();  
  
        myMobil.maju();    // Output: Mobil maju.  
        myMobil.berhenti(); // Output: Kendaraan berhenti.  
    }  
}
```



Interface

Interface

Interface berbentuk abstract secara implisit, baik dari keyword Interface nya, maupun method-method didalamnya.

Jika Class diibaratkan sebagai sebuah Blueprint, Interface di sisi lain diibaratkan sebagai sebuah Contract yang wajib dipatuhi (dalam arti diimplementasikan).

Method-method pada interface secara implisit memiliki akses modifier public.

Perhatikan penulisan interface serta pengimplementasiannya pada gambar disamping

```
public interface Kendaraan {  
    public void maju();  
    public void berhenti();  
}
```

```
// Mobil wajib mengimplementasikan semua method  
// dari interface Kendaraan  
public class Mobil implements Kendaraan {  
    private String merek;  
  
    public Mobil(String merek) {  
        this.merek = merek;  
    }  
  
    @Override  
    public void maju() {  
        System.out.println("Mobil " + merek + " maju.");  
    }  
  
    @Override  
    public void berhenti() {  
        System.out.println("Mobil " + merek + " berhenti.");  
    }  
}
```

Interface (lanjutan)

Suatu class dapat mengimplementasi lebih dari 1 Interface.

```
public class MobilListrik implements Kendaraan, Baterai {
    private String merek;

    public MobilListrik(String merek) {
        this.merek = merek;
    }

    @Override
    public void maju() {
        System.out.println("Mobil " + merek + " maju.");
    }

    @Override
    public void isiDaya() {
        System.out.println("Baterai mobil " + merek + " sudah");
    }
}
```

Interface dapat dapat meng-extend lebih dari 1 Interface lainnya.

```
public interface KendaraanOtonom extends Kendaraan, Navigasi {
    public void kemudiOtomatis();
}
```

03

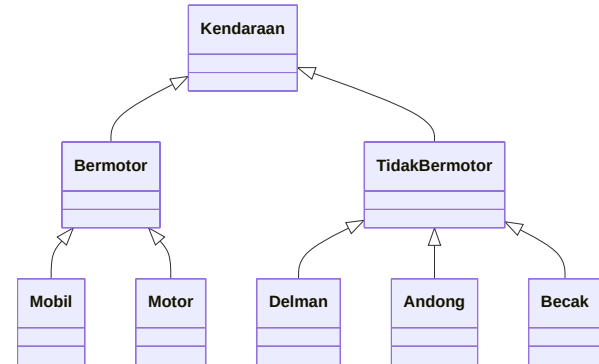
Polimorfisme

Polimorfisme

Polimorfisme adalah kemampuan untuk mempunyai beberapa bentuk class yang berbeda. Polimorfisme ini terjadi pada saat suatu objek bertipe **parent class**, akan tetapi **pemanggilan constructornya melalui child class/sub class**.

Contoh:

```
Kendaraan bmw = new Mobil();
```



Jenis-jenis Polimorfisme

Compile-time (Static Polymorphism)

Static Polymorphism dicapai melalui **overloading method**.

- Nama method sama, tetapi parameternya berbeda.
- Ditentukan saat waktu kompilasi (*compile-time*).

```
class Kalkulator {  
    // Metode 1: 2 parameter int  
    int tambah(int a, int b) {  
        return a + b;  
    }  
  
    // Metode 2: 3 parameter int  
    int tambah(int a, int b, int c) {  
        return a + b + c;  
    }  
  
    // Metode 3: 2 parameter double  
    double tambah(double a, double b) {  
        return a + b;  
    }  
}
```

Jenis-jenis Polimorfisme

Runtime (Dynamic Polymorphism)

Dynamic Polymorphism dicapai melalui **overriding method**.

- Subclass mengubah implementasi metode dari superclass (nama & parameter sama persis).
- Ditentukan saat program berjalan (*runtime*).

```
public class Main {  
    public static void main(String[] args) {  
        Hewan hewan1 = new Kucing();  
        Hewan hewan2 = new Anjing();  
  
        hewan1.bersuara(); // Output: Meow!  
        hewan2.bersuara(); // Output: Guk guk!  
  
        hewan1.lompat(); // Error!  
    }  
}
```

```
class Hewan {  
    void bersuara() {  
        System.out.println("Hewan membuat suara");  
    }  
}
```

```
class Kucing extends Hewan {  
    @Override  
    void bersuara() {  
        System.out.println("Meow!");  
    }  
  
    void lompat() {  
        System.out.println("Jleng!!");  
    }  
}
```

```
class Anjing extends Hewan {  
    @Override  
    void bersuara() {  
        System.out.println("Guk guk!");  
    }  
}
```

Perbandingan

Deklarasi Objek: Biasa vs Polimorfisme

Aspek	<code>Kucing hewan1 = new Kucing();</code>	<code>Hewan hewan1 = new Kucing();</code>
Tipe Referensi	Spesifik (<i>Subclass</i>)	Fleksibel / General (<i>Superclass</i>)
Akses Metode	Bisa panggil semua metode di <code>Hewan</code> dan metode khusus <code>Kucing</code> .	Hanya bisa panggil metode yang ada di kelas <code>Hewan</code> .
Penggunaan Polimorfisme	Tidak menggunakan polimorfisme.	Menggunakan polimorfisme (<i>Upcasting</i>).

Polimorfisme (lanjutan)

```
public class Main {  
    public static void main(String[] args) {  
  
        Kucing kucing = new Kucing();  
        Anjing anjing = new Anjing();  
        Kelinci kelinci = new Kelinci();  
  
        Hewan[] hewanArray = {kucing, anjing, kelinci};  
  
        for (Hewan h : hewanArray) {  
            h.bersuara();  
        }  
    }  
}
```

04

Operator Instance of

Instance Of

Operator instanceof digunakan untuk mengecek apakah objek adalah sebuah instance dari sebuah Object type (class / subclass / interface). Biasa digunakan ketika menggunakan konsep polimorfisme.

Contoh sederhana:

```
public class TestClass{  
    public static void main(String[] args) {  
        TestClass test = new TestClass();  
        System.out.println(test instanceof TestClass); // True  
    }  
}
```

Instance Of

Contoh lain ketika ada sebuah Class Cat extend Class Animal:

```
public class TestClass{  
    public static void main(String[] args) {  
        Cat kucing = new Cat();  
        System.out.println(kucing instanceof Animal); // True  
    }  
}
```

05

Casting

Casting

Casting merupakan aksi ketika kita mengubah suatu tipe data menjadi tipe data yang lain.

Misal hirarki: **Food → Fruit → Apple**

Upcasting:

```
Fruit fruit = new Apple();
```

Memasukkan objek Apel ke dalam wadah bertuliskan Buah.

Apa output dari `fruit instanceof Fruit`?

Downcasting:

```
Apple castedApple = (Apple) fruit;
```

Melihat kembali isi wadah Buah tersebut sebagai Apel.

Apa output dari `castedApple instanceof Fruit`?

Latihan Minggu 4

Semua bisa karena terbiasa

https://drive.google.com/drive/folders/1HTPcjFmmlG7mxOjzMBDszU-QxxwBB3gm?usp=drive_link

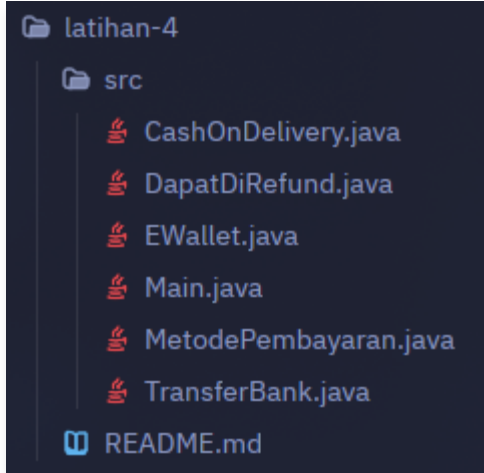
Instruksi Latihan

Sebuah platform e-commerce memerlukan modul pemrosesan pembayaran.

Setiap metode pembayaran memiliki mekanisme transaksi yang berbeda-beda, dan beberapa metode tertentu mendukung pengembalian dana (refund).

TransferBank.java

```
1  // TODO:
2  // 1. Warisi (extends) abstract class MetodePembayaran
3  // 2. Implementasikan (implements) interface DapatDiRefund
4  public class TransferBank {
5      private String nomorRekening;
6
7      public TransferBank(String namaMetode, double saldo, String nomorRekening) {
8          // TODO: Panggil constructor superclass menggunakan super(...)
9          this.nomorRekening = nomorRekening;
10     }
```



Assignment!

Kurang seru praktikum tanpa tugas :D

Silahkan cek Classmoji masing-masing, tugas dapat diakses setelah dipublish dan diinformasikan oleh asprak kelas masing-masing.

Instruksi Assignment

Dalam sebuah sistem Smart Home, terdapat Central Hub yang mengatur berbagai macam peralatan elektronik. Setiap perangkat memiliki konsumsi daya listrik dasar dan fungsionalitas unik. Sebagian perangkat dapat dihubungkan ke jaringan Wi-Fi, sebagian dapat diatur intensitasnya, dan beberapa perangkat hanya memiliki sakelar hidup/mati biasa.

Silahkan clone repositori terkait, tidak perlu menulis kode dari nol. Implementasikan konsep-konsep yang sudah dipelajari untuk melengkapi kode-kode yang belum lengkap. Kerjakan hanya bagian kode yang bertuliskan TODO.

```
// TODO:  
// 1. Warisi (extends) abstract class PerangkatElektronik  
// 2. Implementasikan (implements) interface DapatTerhubungWiFi  
// PERHATIAN: KameraKeamanan TIDAK mengimplementasikan interface DapatDisesuaikan!  
public class KameraKeamanan {  
    private int kapasitasMemoriGB;  
    private boolean sedangMerekam;  
  
    public KameraKeamanan(String idPerangkat, String namaPerangkat, double dayaWatt)  
    // TODO: Panggil constructor superclass menggunakan super(...)  
    {  
        this.kapasitasMemoriGB = kapasitasMemoriGB;  
        this.sedangMerekam = true;  
    }  
}
```

Deadline Pengumpulan

Kelas A:

21 September 2026

Kelas B:

20 September 2026

Kelas C:

22 September 2026

Waktu yang dilihat adalah waktu last commit. Jika ada yang commit melewati deadline walaupun sudah commit sebelumnya akan dianggap terlambat.

Terima Kasih!

Do you have any questions? Please use respective class discussion channel on Discord.

Key of success adalah kunci kesuksesan 🔥🔥🔥